

Travaux Pratiques : OpenSSL et Cryptographie

Pr. RAISS El Fenni Mohammed

2025-2026

1 Introduction

Ce TP a pour objectif de comprendre et de se familiariser avec les notions cryptographiques à travers l'utilisation d'OpenSSL. Nous explorerons le chiffrement symétrique et asymétrique, le hachage, la signature numérique et la gestion des certificats.

2 Codage Base64

L'encodage Base64 est une méthode couramment utilisée pour représenter des données binaires sous forme de texte ASCII. C'est un système d'encodage qui convertit chaque bloc de 3 octets (24 bits) en une séquence de 4 caractères ASCII.

1. Créer un fichier texte contenant un mot de passe arbitraire.

Cette étape consiste à générer un fichier texte contenant un mot de passe fictif.

```
echo "MotDePasse123" > fich_clair
```

Afficher le contenu du fichier pour vérification :

```
cat fich_clair
```

2. Encoder le fichier en Base64.

L'encodage Base64 convertit le fichier en une représentation textuelle sécurisée.

```
openssl base64 -in fich_clair -out fich_64
```

Afficher le contenu du fichier encodé :

```
cat fich_64
```

3. Décoder le fichier encodé.

Cette étape permet de restaurer le fichier original après l'encodage Base64.

```
openssl base64 -d -in fich_64 -out fich_decode
```

Vérifier que le contenu décodé est identique au fichier original :

```
cat fich_decode
```

3 Chiffrement Symétrique avec AES

Le chiffrement symétrique utilise une même clé pour chiffrer et déchiffrer les données. L'algorithme AES est largement utilisé pour garantir la confidentialité des données.

3.1 Chiffrement avec AES

L'objectif est de chiffrer un fichier avec AES en mode CBC (Cipher Block Chaining), qui améliore la sécurité en introduisant un vecteur d'initialisation (IV).

1.1 Création d'un fichier à chiffrer :

```
echo "Message_confidentiel" > fichier_clair.txt
```

1.2 Chiffrement avec AES-256-CBC :

```
openssl enc -aes-256-cbc -pbkdf2 -salt -k motdepasse -in fichier_clair.txt -  
out fichier_chiffre.aes
```

Cette commande :

- Utilise l'algorithme AES en mode CBC avec une clé de 256 bits.
- Applique PBKDF2 (Password-Based Key Derivation Function 2) pour renforcer la clé issue du mot de passe.
- Ajoute un sel pour éviter que deux fichiers identiques chiffrés avec le même mot de passe aient le même chiffrement.
- Chiffre le fichier 'fichier_clair.txt' et génère 'fichier_chiffre.aes'.

1.3 Vérification du fichier chiffré :

```
cat fichier_chiffre.aes
```

3.2 Déchiffrement avec AES

Le déchiffrement nécessite l'utilisation du même mot de passe utilisé pour le chiffrement.

2.1 Déchiffrement du fichier chiffré :

```
openssl enc -d -aes-256-cbc -pbkdf2 -in fichier_chiffre.aes -out  
fichier_dechiffre.txt -k motdepasse
```

Cette commande :

- Utilise le même algorithme et mode de chiffrement.
- Reconvertit "fichier_chiffre.aes" en 'fichier_dechiffre.txt' en utilisant le bon mot de passe.

2.2 Vérification du fichier déchiffré :

```
cat fichier_dechiffre.txt
```

4 RSA avec OpenSSL

L'algorithme RSA est utilisé pour sécuriser les communications, en assurant la confidentialité et l'authenticité des données.

4.1 Génération d'une paire de clés

La première étape consiste à générer une paire de clés RSA, qui comprend une clé privée et une clé publique.

1. Générer une clé privée RSA :

```
openssl genrsa -out maCle.pem 1024
```

Cette commande génère une clé privée RSA de 1024 bits.

2. Extraire la clé publique à partir de la clé privée :

```
openssl rsa -in maCle.pem -pubout -out smaClePublique.pem
```

Cela permet d'obtenir la clé publique à partir de la clé privée générée.

3. Vérifier les clés générées :

```
cat maCle.pem  
cat maClePublique.pem
```

4.2 Visualisation des clés RSA

Une fois les clés générées, nous pouvons afficher leurs détails pour vérifier leur structure et leur contenu.

1. Affichage de la clé privée : Cette commande permet d'afficher les informations détaillées de la clé privée, y compris le module, l'exposant public et les coefficients associés.

```
openssl rsa -in maCle.pem -text -noout
```

2. Affichage de la clé publique : Une fois la clé publique extraite, nous pouvons également visualiser son contenu.

```
openssl rsa -in maClePublique.pem -pubin -text -noout
```

4.3 Chiffrement d'un fichier de clés RSA

Il est risqué de conserver une clé privée en clair. Pour protéger celle-ci, nous allons la chiffrer avec un mot de passe.

```
openssl rsa -in maCle.pem -des3 -out maCle_chiffre.pem
```

Vérifier que la clé privée est maintenant protégée :

```
cat maCle_chiffre.pem
```

4.4 Exportation de la partie publique

La clé publique peut être extraite de la clé privée pour être partagée en toute sécurité.

```
openssl rsa -in maCle_chiffre.pem -pubout -out maClePublique.pem
```

Afficher le contenu du fichier de la clé publique :

```
cat maClePublique.pem
```

4.5 Chiffrement et Déchiffrement de Données avec RSA

L'algorithme RSA permet de chiffrer et de déchiffrer des données en utilisant une **clé publique** pour le chiffrement et une **clé privée** pour le déchiffrement. Cette méthode garantit la **confidentialité** des échanges.

1. Préparation : Avant de procéder au chiffrement, nous allons créer un fichier contenant les données à protéger.

```
echo "Message_confidentiel" > fichier_entree
```

2. Chiffrement : Le fichier est chiffré à l'aide de la **clé publique** afin que seul le détenteur de la **clé privée** puisse le déchiffrer.

```
openssl rsautl -encrypt -in fichier_entree -inkey maClePublique.pem -pubin -  
out fichier_sortie
```

Vérifier que le fichier a bien été chiffré : L'affichage du contenu du fichier chiffré montrera une séquence de caractères illisibles, indiquant que les données sont bien protégées.

```
cat fichier_sortie
```

3. Déchiffrement : Le détenteur de la **clé privée** peut récupérer les données d'origine en utilisant la commande suivante :

```
openssl rsautl -decrypt -in fichier_sortie -inkey maCle.pem -out  
fichier_dechiffre
```

Vérifier le contenu du fichier déchiffré : Après déchiffrement, le fichier doit contenir le même message que le fichier original.

```
cat fichier_dechiffre
```

5 Signature des Fichiers

Signer un fichier permet de garantir son **intégrité** et l'**authenticité** de son auteur.

- **L'intégrité** assure que le fichier n'a pas été modifié après sa signature.
- **L'authenticité** permet de vérifier l'identité du signataire en confirmant que la signature provient bien de la clé privée associée.

5.1 Génération de la signature

Nous allons d'abord générer une **empreinte cryptographique** du fichier, puis la signer avec la **clé privée**.

```
openssl dgst -sha256 -sign maCle.pem -out signature fichier.txt
```

Cette commande :

- Génère une empreinte SHA-256 du fichier `fichier.txt`.
- Chiffre cette empreinte en utilisant la **clé privée** `maCle.pem`.
- Stocke le résultat signé dans `signature`.

Vérification de la signature générée : Pour s'assurer que la signature a bien été créée, affichons son contenu en hexadécimal :

```
xxd signature
```

5.2 Vérification de la signature

Pour vérifier que le fichier signé est valide, nous devons :

- Déchiffrer la signature avec la **clé publique**.
- Comparer l'empreinte obtenue avec celle du fichier d'origine.

Décryptage de la signature : On utilise la clé publique pour retrouver l’empreinte d’origine signée.

```
openssl dgst -sha256 -verify maClePublique.pem -signature signature fichier.txt
```

Cette commande :

- Déchiffre la signature avec la **clé publique** `maClePublique.pem`.
- Compare l’empreinte obtenue avec celle calculée sur le fichier d’origine.
- Affiche **Verified OK** si la signature est correcte.

Comparaison manuelle de l’empreinte SHA-256 : Nous pouvons également comparer directement les empreintes :

```
openssl dgst -sha256 fichier.txt
openssl rsautl -verify -in signature -pubin -inkey maClePublique.pem | xxd
```

Si les deux empreintes obtenues sont identiques, la signature est valide et garantit que le fichier n’a pas été altéré.

6 Certificats

Un certificat numérique permet d’associer une identité à une clé publique. Il est utilisé pour garantir l’authenticité des échanges sécurisés, notamment dans les communications SSL/TLS.

6.1 Création d’une requête de certificat (CSR)

Une requête de certificat (Certificate Signing Request - CSR) est nécessaire pour demander un certificat auprès d’une autorité de certification.

Génération de la requête de certificat :

```
openssl req -new -key maCle.pem -out csr.pem
```

Cette commande :

- Génère une requête de certificat pour la clé privée existante.
- Demande des informations sur le propriétaire du certificat (nom, organisation, email, etc.).
- Stocke la requête dans le fichier `csr.pem`.

Vérification du contenu de la CSR :

```
cat csr.pem
```

Le fichier contient des balises similaires à :

```
-----BEGIN CERTIFICATE REQUEST-----
...
-----END CERTIFICATE REQUEST-----
```

6.2 Demande de signature de certificat

Une fois la requête CSR générée, nous pouvons la signer avec notre propre clé privée pour créer un certificat auto-signé. Ce certificat peut être utilisé pour des tests ou des connexions sécurisées.

Signature de la requête CSR pour générer un certificat :

```
openssl x509 -req -days 365 -in csr.pem -signkey maCle.pem
-out monCertif.pem
```

Cette commande :

- Prend en entrée la requête CSR `csr.pem`.
- Utilise la clé privée `maCle.pem` pour signer le certificat.
- Génère un certificat valide pendant 365 jours.
- Stocke le certificat final dans `monCertif.pem`.

Vérification du fichier de certificat :

```
cat monCertif.pem
```

Le fichier doit contenir des balises similaires à :

```
-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----
```

7 Scénario : Communication Sécurisée entre Deux Utilisateurs

Dans cette section, nous mettons en place un scénario où deux utilisateurs, A et B, souhaitent communiquer en toute sécurité en utilisant la cryptographie asymétrique.

Objectif :

- L'utilisateur A veut envoyer un message chiffré à B.
- Le message doit être chiffré avec la clé publique de B.
- L'utilisateur A doit posséder un certificat contenant la clé publique de B.
- L'utilisateur B utilisera sa clé privée pour déchiffrer le message.

7.1 1. Génération de la paire de clés pour B

L'utilisateur B doit d'abord générer sa propre paire de clés RSA.

```
openssl genpkey -algorithm RSA -out cle_privee_B.pem -pkeyopt rsa_keygen_bits
:2048
openssl rsa -in cle_privee_B.pem -pubout -out cle_publique_B.pem
```

Question 1 : Pourquoi est-il important que la clé privée de B reste confidentielle ?

7.2 2. Création et signature du certificat de B

Pour garantir l'authenticité de la clé publique de B, celui-ci génère une requête de certificat (CSR) et la signe.

```
openssl req -new -key cle_privee_B.pem -out demande_certificat_B.csr
openssl x509 -req -days 365 -in demande_certificat_B.csr -signkey cle_privee_B
.pem -out certificat_B.pem
```

Question 2 : Quelle est la différence entre un certificat auto-signé et un certificat délivré par une autorité de certification ?

7.3 3. Vérification et remise du certificat à A

L'utilisateur A doit récupérer le certificat de B et l'inspecter pour vérifier son authenticité.

```
openssl x509 -in certificat_B.pem -text -noout
```

Question 3 : Pourquoi A ne peut-il pas chiffrer un message directement avec la clé privée de B ?

7.4 4. A chiffre un message avec la clé publique de B

L'utilisateur A veut envoyer un message sécurisé à B. Il chiffre son message avec la clé publique contenue dans le certificat de B.

```
echo "Bonjour" > message_clair.txt
openssl rsautl -encrypt -inkey certificat_B.pem -pubin -in message_clair.txt -
out message_chiffre.txt
```

Question 4 : Pourquoi le chiffrement est-il effectué avec la clé publique de B et non avec la clé privée de A ?

7.5 5. B déchiffre le message avec sa clé privée

À la réception du message, B utilise sa clé privée pour récupérer le message original.

```
openssl rsautl -decrypt -inkey cle_privee_B.pem -in message_chiffre.txt -out
message_dechiffre.txt
cat message_dechiffre.txt
```

Question 5 : Que se passe-t-il si une personne malveillante intercepte le message chiffré ? Peut-elle le lire ?

Question de reflexion : Ce scénario montre comment deux utilisateurs peuvent sécuriser leurs communications en utilisant la cryptographie asymétrique. A utilise la clé publique de B pour garantir que seul B peut déchiffrer le message, assurant ainsi la confidentialité de l'échange. Quelles seraient les étapes supplémentaires nécessaires afin de garantir non seulement la confidentialité, mais aussi l'authenticité de l'expéditeur ?